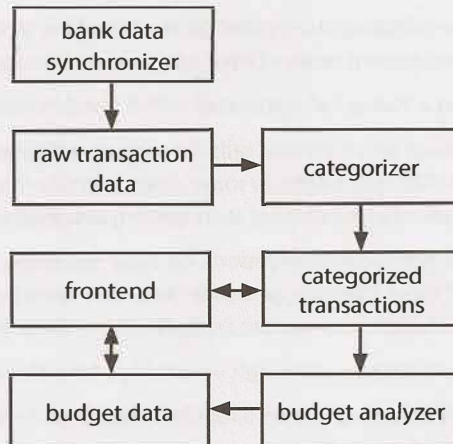


Step 3: Draw the Major Components

The most naive system would be one that pulls bank data on each login, categorizes all the data, and then analyzes the user's budget. This wouldn't quite fit the requirements, though, as we want email notifications on particular events.

We can do a bit better.



With this basic architecture, the bank data is pulled at periodic times (hourly or daily). The frequency may depend on the behavior of the users. Less active users may have their accounts checked less frequently.

Once new data arrives, it is stored in some list of raw, unprocessed transactions. This data is then pushed to the categorizer, which assigns each transaction to a category and stores these categorized transactions in another datastore.

The budget analyzer pulls in the categorized transactions, updates each user's budget per category, and stores the user's budget.

The frontend pulls data from both the categorized transactions datastore as well as from the budget datastore. Additionally, a user could also interact with the frontend by changing the budget or the categorization of their transactions.

Step 4: Identify the Key Issues

We should now reflect on what the major issues here might be.

This will be a very data-heavy system. We want it to feel snappy and responsive, though, so we'll want as much processing as possible to be asynchronous.

We will almost certainly want at least one task queue, where we can queue up work that needs to be done. This work will include tasks such as pulling in new bank data, re-analyzing budgets, and categorizing new bank data. It would also include re-trying tasks that failed.

These tasks will likely have some sort of priority associated with them, as some need to be performed more often than others. We want to build a task queue system that can prioritize some task types over others, while still ensuring that all tasks will be performed eventually. That is, we wouldn't want a low priority task to essentially "starve" because there are always higher priority tasks.

One important part of the system that we haven't yet addressed will be the email system. We could use a task to regularly crawl user's data to check if they're exceeding their budget, but that means checking every

single user daily. Instead, we'll want to queue a task whenever a transaction occurs that potentially exceeds a budget. We can store the current budget totals by category to make it easy to understand if a new transaction exceeds the budget.

We should also consider incorporating the knowledge (or assumption) that a system like this will probably have a large number of inactive users—users who signed up once and then haven't touched the system since. We may want to either remove them from the system entirely or deprioritize their accounts. We'll want some system to track their account activity and associate priority with their accounts.

The biggest bottleneck in our system will likely be the massive amount of data that needs to be pulled and analyzed. We should be able to fetch the bank data asynchronously and run these tasks across many servers. We should drill a bit deeper into how the categorizer and budget analyzer work.

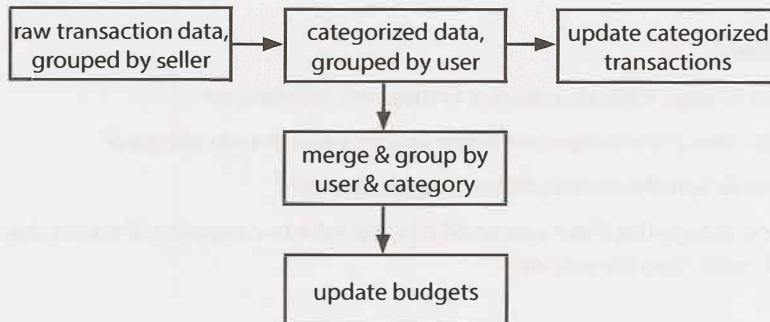
Categorizer and Budget Analyzer

One thing to note is that transactions are not dependent on each other. As soon as we get a transaction for a user, we can categorize it and integrate this data. It might be inefficient to do so, but it won't cause any inaccuracies.

Should we use a standard database for this? With lots of transactions coming in at once, that might not be very efficient. We certainly don't want to do a bunch of joins.

It may be better instead to just store the transactions to a set of flat text files. We assumed earlier that the categorizations are based on the seller's name alone. If we're assuming a lot of users, then there will be a lot of duplicates across the sellers. If we group the transaction files by seller's name, we can take advantage of these duplicates.

The categorizer can do something like this:



It first gets the raw transaction data, grouped by seller. It picks the appropriate category for the seller (which might be stored in a cache for the most common sellers), and then applies that category to all those transactions.

After applying the category, it re-groups all the transactions by user. Then, those transactions are inserted into the datastore for this user.

before categorizer	after categorizer
amazon/ user121,\$5.43,Aug 13 user922,\$15.39,Aug 27 ...	user121/ amazon,shopping,\$5.43,Aug 13 ...
comcast/ user922,\$9.29,Aug 24 user248,\$40.13,Aug 18 ...	user922/ amazon,shopping,\$15.39,Aug 27 comcast,utilities,\$9.29,Aug 24 ...
	user248/ comcast,utilities,\$40.13,Aug 18 ...

Then, the budget analyzer comes in. It takes the data grouped by user, merges it across categories (so all Shopping tasks for this user in this timespan are merged), and then updates the budget.

Most of these tasks will be handled in simple log files. Only the final data (the categorized transactions and the budget analysis) will be stored in a database. This minimizes writing and reading from the database.

User Changing Categories

The user might selectively override particular transactions to assign them to a different category. In this case, we would update the datastore for the categorized transactions. It would also signal a quick recomputation of the budget to decrement the item from the old category and increment the item in the other category.

We could also just recompute the budget from scratch. The budget analyzer is fairly quick as it just needs to look over the past few weeks of transactions for a single user.

Follow Up Questions

- How would this change if you also needed to support a mobile app?
- How would you design the component which assigns items to each category?
- How would you design the recommended budgets feature?
- How would you change this if the user could develop rules to categorize all transactions from a particular seller differently than the default?

9.8 Pastebin: Design a system like Pastebin, where a user can enter a piece of text and get a randomly generated URL for public access.

pg 145

SOLUTION

We can start with clarifying the specifics of this system.

Step 1: Scope the Problem

- The system does not support user accounts or editing documents.
- The system tracks analytics of how many times each page is accessed.
- Old documents get deleted after not being accessed for a sufficiently long period of time.
- While there isn't true authentication on accessing documents, users should not be able to "guess" docu-

ment URLs easily.

- The system has a frontend as well as an API.
- The analytics for each URL can be accessed through a “stats” link on each page. It is not shown by default, though.

Step 2: Make Reasonable Assumptions

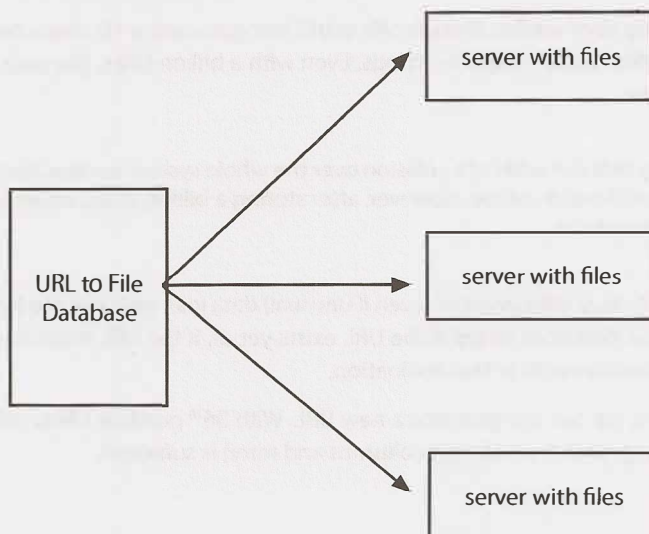
- The system gets heavy traffic and contains many millions of documents.
- Traffic is not equally distributed across documents. Some documents get much more access than others.

Step 3: Draw the Major Components

We can sketch out a simple design. We’ll need to keep track of URLs and the files associated with them, as well as analytics for how often the files have been accessed.

How should we store the documents? We have two options: we can store them in a database or we can store them on a file. Since the documents can be large and it’s unlikely we need searching capabilities, storing them on a file is probably the better choice.

A simple design like this might work well:



Here, we have a simple database that looks up the location (server and path) of each file. When we have a request for a URL, we look up the location of the URL within the datastore and then access the file.

Additionally, we will need a database that tracks analytics. We can do this with a simple datastore that adds each visit (including timestamp, IP address, and location) as a row in a database. When we need to access the stats of each visit, we pull the relevant data in from this database.

Step 4: Identify the Key Issues

The first issue that comes to mind is that some documents will be accessed much more frequently than others. Reading data from the filesystem is relatively slow compared with reading from data in memory. Therefore, we probably want to use a cache to store the most recently accessed documents. This will ensure

that items accessed very frequently (or very recently) will be quickly accessible. Since documents cannot be edited, we will not need to worry about invalidating this cache.

We should also potentially consider sharding the database. We can shard it using some mapping from the URL (for example, the URL's hash code modulo some integer), which will allow us to quickly locate the database which contains this file.

In fact, we could even take this a step further. We could skip the database entirely and just let a hash of the URL indicate which server contains the document. The URL itself could reflect the location of the document. One potential issue from this is that if we need to add servers, it could be difficult to redistribute the documents.

Generating URLs

We have not yet discussed how to actually generate the URLs. We probably do not want a monotonically increasing integer value, as this would be easy for a user to “guess.” We want URLs to be difficult to access without being provided the link.

One simple path is to generate a random GUID (e.g., 5d50e8ac-57cb-4a0d-8661-bcdee2548979). This is a 128-bit value that, while not strictly guaranteed to be unique, has low enough odds of a collision that we can treat it as unique. The drawback of this plan is that such a URL is not very “pretty” to the user. We could hash it to a smaller value, but then that increases the odds of collision.

We could do something very similar, though. We could just generate a 10-character sequence of letters and numbers, which gives us 36^{10} possible strings. Even with a billion URLs, the odds of a collision on any specific URL are very low.

This is not to say that the odds of a collision over the whole system are low. They are not. Any one specific URL is unlikely to collide. However, after storing a billion URLs, we are very likely to have a collision at some point.

Assuming that we aren't okay with periodic (even if unusual) data loss, we'll need to handle these collisions. We can either check the datastore to see if the URL exists yet or, if the URL maps to a specific server, just detect whether a file already exists at the destination.

When a collision occurs, we can just generate a new URL. With 36^{10} possible URLs, collisions would be rare enough that the lazy approach here (detect collisions and retry) is sufficient.

Analytics

The final component to discuss is the analytics piece. We probably want to display the number of visits, and possibly break this down by location or time.

We have two options here:

- Store the raw data from each visit.
- Store just the data we know we'll use (number of visits, etc.).

You can discuss this with your interviewer, but it probably makes sense to store the raw data. We never know what features we'll add to the analytics down the road. The raw data allows us flexibility.

This does not mean that the raw data needs to be easily searchable or even accessible. We can just store a log of each visit in a file, and back this up to other servers.

One issue here is that this amount of data could be substantial. We could potentially reduce the space usage considerably by storing data only probabilistically. Each URL would have a `storage_probability` associated with it. As the popularity of a site goes up, the `storage_probability` goes down. For example, a popular document might have data logged only one out of every ten times, at random. When we look up the number of visits for the site, we'll need to adjust the value based on the probability (for example, by multiplying it by 10). This will of course lead to a small inaccuracy, but that may be acceptable.

The log files are not designed to be used frequently. We will want to also store this precomputed data in a datastore. If the analytics just displays the number of visits plus a graph over time, this could be kept in a separate database.

URL	Month and Year	Visits
12ab31b92p	December 2013	242119
12ab31b92p	January 2014	429918
...	...	...

Every time a URL is visited, we can increment the appropriate row and column. This datastore can also be sharded by the URL.

As the stats are not listed on the regular pages and would generally be of less interest, it should not face as heavy of a load. We could still cache the generated HTML on the frontend servers, so that we don't continuously reaccess the data for the most popular URLs.

Follow-Up Questions

- How would you support user accounts?
- How would you add a new piece of analytics (e.g., referral source) to the stats page?
- How would your design change if the stats were shown with each document?

10

Solutions to Sorting and Searching

10.1 Sorted Merge: You are given two sorted arrays, A and B, where A has a large enough buffer at the end to hold B. Write a method to merge B into A in sorted order.

pg 149

SOLUTION

Since we know that A has enough buffer at the end, we won't need to allocate additional space. Our logic should involve simply comparing elements of A and B and inserting them in order, until we've exhausted all elements in A and in B.

The only issue with this is that if we insert an element into the front of A, then we'll have to shift the existing elements backwards to make room for it. It's better to insert elements into the back of the array, where there's empty space.

The code below does just that. It works from the back of A and B, moving the largest elements to the back of A.

```
1 void merge(int[] a, int[] b, int lastA, int lastB) {
2     int indexA = lastA - 1; /* Index of last element in array a */
3     int indexB = lastB - 1; /* Index of last element in array b */
4     int indexMerged = lastB + lastA - 1; /* end of merged array */
5
6     /* Merge a and b, starting from the last element in each */
7     while (indexB >= 0) {
8         /* end of a is > than end of b */
9         if (indexA >= 0 && a[indexA] > b[indexB]) {
10             a[indexMerged] = a[indexA]; // copy element
11             indexA--;
12         } else {
13             a[indexMerged] = b[indexB]; // copy element
14             indexB--;
15         }
16         indexMerged--; // move indices
17     }
18 }
```

Note that you don't need to copy the contents of A after running out of elements in B. They are already in place.

10.2 Group Anagrams: Write a method to sort an array of strings so that all the anagrams are next to each other.

pg 150

SOLUTION

This problem asks us to group the strings in an array such that the anagrams appear next to each other. Note that no specific ordering of the words is required, other than this.

We need a quick and easy way of determining if two strings are anagrams of each other. What defines if two words are anagrams of each other? Well, anagrams are words that have the same characters but in different orders. It follows then that if we can put the characters in the same order, we can easily check if the new words are identical.

One way to do this is to just apply any standard sorting algorithm, like merge sort or quick sort, and modify the comparator. This comparator will be used to indicate that two strings which are anagrams of each other are equivalent.

What's the easiest way of checking if two words are anagrams? We could count the occurrences of the distinct characters in each string and return `true` if they match. Or, we could just sort the string. After all, two words which are anagrams will look the same once they're sorted.

The code below implements the comparator.

```
1 class AnagramComparator implements Comparator<String> {
2     public String sortChars(String s) {
3         char[] content = s.toCharArray();
4         Arrays.sort(content);
5         return new String(content);
6     }
7
8     public int compare(String s1, String s2) {
9         return sortChars(s1).compareTo(sortChars(s2));
10    }
11 }
```

Now, just sort the arrays using this `compareTo` method instead of the usual one.

```
12 Arrays.sort(array, new AnagramComparator());
```

This algorithm will take $O(n \log(n))$ time.

This may be the best we can do for a general sorting algorithm, but we don't actually need to fully sort the array. We only need to *group* the strings in the array by anagram.

We can do this by using a hash table which maps from the sorted version of a word to a list of its anagrams. So, for example, `acre` will map to the list `{acre, race, care}`. Once we've grouped all the words into these lists by anagram, we can then put them back into the array.

The code below implements this algorithm.

```
1 void sort(String[] array) {
2     HashMapList<String, String> mapList = new HashMapList<String, String>();
3
4     /* Group words by anagram */
5     for (String s : array) {
6         String key = sortChars(s);
7         mapList.put(key, s);
8     }
```

```
9
10 / *Convert hash table to array */
11 int index = 0;
12 for (String key : mapList.keySet()) {
13     ArrayList<String> list = mapList.get(key);
14     for (String t : list) {
15         array[index] = t;
16         index++;
17     }
18 }
19 }
20
21 String sortChars(String s) {
22     char[] content = s.toCharArray();
23     Arrays.sort(content);
24     return new String(content);
25 }
26
27 / *HashMapList<String, Integer> is a HashMap that maps from Strings to
28 * ArrayList<Integer>. See appendix for implementation. */
```

You may notice that the algorithm above is a modification of bucket sort.

10.3 Search in Rotated Array: Given a sorted array of n integers that has been rotated an unknown number of times, write code to find an element in the array. You may assume that the array was originally sorted in increasing order.

EXAMPLE

Input: find 5 in {15, 16, 19, 20, 25, 1, 3, 4, 5, 7, 10, 14}

Output: 8 (the index of 5 in the array)

pg 150

SOLUTION

If this problem smells like binary search to you, you're right!

In classic binary search, we compare x with the midpoint to figure out if x belongs on the left or the right side. The complication here is that the array is rotated and may have an inflection point. Consider, for example, the following two arrays:

Array1: {10, 15, 20, 0, 5}

Array2: {50, 5, 20, 30, 40}

Note that both arrays have a midpoint of 20, but 5 appears on the left side of one and on the right side of the other. Therefore, comparing x with the midpoint is insufficient.

However, if we look a bit deeper, we can see that one half of the array must be ordered normally (in increasing order). We can therefore look at the normally ordered half to determine whether we should search the left or right half.

For example, if we are searching for 5 in Array1, we can look at the left element (10) and middle element (20). Since $10 < 20$, the left half must be ordered normally. And, since 5 is not between those, we know that we must search the right half.

In `Array2`, we can see that since $50 > 20$, the right half must be ordered normally. We turn to the middle (20) and right (40) element to check if 5 would fall between them. The value 5 would not; therefore, we search the left half.

The tricky condition is if the left and the middle are identical, as in the example array {2, 2, 2, 3, 4, 2}. In this case, we can check if the rightmost element is different. If it is, we can search just the right side. Otherwise, we have no choice but to search both halves.

```

1  int search(int a[], int left, int right, int x) {
2      int mid = (left + right) / 2;
3      if (x == a[mid]) { // Found element
4          return mid;
5      }
6      if (right < left) {
7          return -1;
8      }
9
10     /* Either the left or right half must be normally ordered. Find out which side
11      * is normally ordered, and then use the normally ordered half to figure out
12      * which side to search to find x. */
13     if (a[left] < a[mid]) { // Left is normally ordered.
14         if (x >= a[left] && x < a[mid]) {
15             return search(a, left, mid - 1, x); // Search left
16         } else {
17             return search(a, mid + 1, right, x); // Search right
18         }
19     } else if (a[mid] < a[right]) { // Right is normally ordered.
20         if (x > a[mid] && x <= a[right]) {
21             return search(a, mid + 1, right, x); // Search right
22         } else {
23             return search(a, left, mid - 1, x); // Search left
24         }
25     } else if (a[left] == a[mid]) { // Left or right half is all repeats
26         if (a[mid] != a[right]) { // If right is different, search it
27             return search(a, mid + 1, right, x); // search right
28         } else { // Else, we have to search both halves
29             int result = search(a, left, mid - 1, x); // Search left
30             if (result == -1) {
31                 return search(a, mid + 1, right, x); // Search right
32             } else {
33                 return result;
34             }
35         }
36     }
37     return -1;
38 }

```

This code will run in $O(\log n)$ if all the elements are unique. However, with many duplicates, the algorithm is actually $O(n)$. This is because with many duplicates, we will often have to search both the left and right sides of the array (or subarrays).

Note that while this problem is not conceptually very complex, it is actually very difficult to implement flawlessly. Don't feel bad if you had trouble implementing it without a few bugs. Because of the ease of making off-by-one and other minor errors, you should make sure to test your code very thoroughly.

10.4 Sorted Search, No Size: You are given an array-like data structure `Listy` which lacks a `size` method. It does, however, have an `elementAt(i)` method that returns the element at index `i` in $O(1)$ time. If `i` is beyond the bounds of the data structure, it returns `-1`. (For this reason, the data structure only supports positive integers.) Given a `Listy` which contains sorted, positive integers, find the index at which an element `x` occurs. If `x` occurs multiple times, you may return any index.

pg 150

SOLUTION

Our first thought here should be binary search. The problem is that binary search requires us knowing the length of the list, so that we can compare it to the midpoint. We don't have that here.

Could we compute the length? Yes!

We know that `elementAt` will return `-1` when `i` is too large. We can therefore just try bigger and bigger values until we exceed the size of the list.

But how much bigger? If we just went through the list linearly—1, then 2, then 3, then 4, and so on—we'd wind up with a linear time algorithm. We probably want something faster than this. Otherwise, why would the interviewer have specified the list is sorted?

It's better to back off exponentially. Try 1, then 2, then 4, then 8, then 16, and so on. This ensures that, if the list has length `n`, we'll find the length in at most $O(\log n)$ time.

Why $O(\log n)$? Imagine we start with pointer `q` at `q = 1`. At each iteration, this pointer `q` doubles, until `q` is bigger than the length `n`. How many times can `q` double in size before it's bigger than `n`? Or, in other words, for what value of `k` does $2^k = n$? This expression is equal when $k = \log n$, as this is precisely what $\log$ means. Therefore, it will take $O(\log n)$ steps to find the length.

Once we find the length, we just perform a (mostly) normal binary search. I say "mostly" because we need to make one small tweak. If the mid point is `-1`, we need to treat this as a "too big" value and search left. This is on line 16 below.

There's one more little tweak. Recall that the way we figure out the length is by calling `elementAt` and comparing it to `-1`. If, in the process, the element is bigger than the value `x` (the one we're searching for), we'll jump over to the binary search part early.

```
1  int search(Listy list, int value) {
2      int index = 1;
3      while (list.elementAt(index) != -1 && list.elementAt(index) < value) {
4          index *= 2;
5      }
6      return binarySearch(list, value, index / 2, index);
7  }
8
9  int binarySearch(Listy list, int value, int low, int high) {
10     int mid;
11
12     while (low <= high) {
13         mid = (low + high) / 2;
14         int middle = list.elementAt(mid);
15         if (middle > value || middle == -1) {
16             high = mid - 1;
17         } else if (middle < value) {
```

```

18         low = mid + 1;
19     } else {
20         return mid;
21     }
22 }
23 return -1;
24 }

```

It turns out that not knowing the length didn't impact the runtime of the search algorithm. We find the length in $O(\log n)$ time and then do the search in $O(\log n)$ time. Our overall runtime is $O(\log n)$, just as it would be in a normal array.

10.5 Sparse Search: Given a sorted array of strings that is interspersed with empty strings, write a method to find the location of a given string.

EXAMPLE

Input: ball, {"at", "", "", "", "ball", "", "", "car", "", "", "dad", "", ""}

Output: 4

pg 150

SOLUTION

If it weren't for the empty strings, we could simply use binary search. We would compare the string to be found, `str`, with the midpoint of the array, and go from there.

With empty strings interspersed, we can implement a simple modification of binary search. All we need to do is fix the comparison against `mid`, in case `mid` is an empty string. We simply move `mid` to the closest non-empty string.

The recursive code below to solve this problem can easily be modified to be iterative. We provide such an implementation in the code attachment.

```

1  int search(String[] strings, String str, int first, int last) {
2      if (first > last) return -1;
3      /* Move mid to the middle */
4      int mid = (last + first) / 2;
5
6      /* If mid is empty, find closest non-empty string. */
7      if (strings[mid].isEmpty()) {
8          int left = mid - 1;
9          int right = mid + 1;
10         while (true) {
11             if (left < first && right > last) {
12                 return -1;
13             } else if (right <= last && !strings[right].isEmpty()) {
14                 mid = right;
15                 break;
16             } else if (left >= first && !strings[left].isEmpty()) {
17                 mid = left;
18                 break;
19             }
20             right++;
21             left--;
22         }
23     }

```

```
24
25  /* Check for string, and recurse if necessary */
26  if (str.equals(strings[mid])) { // Found it!
27      return mid;
28  } else if (strings[mid].compareTo(str) < 0) { // Search right
29      return search(strings, str, mid + 1, last);
30  } else { // Search left
31      return search(strings, str, first, mid - 1);
32  }
33 }
34
35 int search(String[] strings, String str) {
36     if (strings == null || str == null || str == "") {
37         return -1;
38     }
39     return search(strings, str, 0, strings.length - 1);
40 }
```

The worst-case runtime for this algorithm is $O(n)$. In fact, it's impossible to have an algorithm for this problem that is better than $O(n)$ in the worst case. After all, you could have an array of all empty strings except for one non-empty string. There is no “smart” way to find this non-empty string. In the worst case, you will need to look at every element in the array.

Careful consideration should be given to the situation when someone searches for the empty string. Should we find the location (which is an $O(n)$ operation)? Or should we handle this as an error?

There's no correct answer here. This is an issue you should raise with your interviewer. Simply asking this question will demonstrate that you are a careful coder.

10.6 Sort Big File: Imagine you have a 20 GB file with one string per line. Explain how you would sort the file.

pg 150

SOLUTION

When an interviewer gives a size limit of 20 gigabytes, it should tell you something. In this case, it suggests that they don't want you to bring all the data into memory.

So what do we do? We only bring part of the data into memory.

We'll divide the file into chunks, which are x megabytes each, where x is the amount of memory we have available. Each chunk is sorted separately and then saved back to the file system.

Once all the chunks are sorted, we merge the chunks, one by one. At the end, we have a fully sorted file.

This algorithm is known as external sort.

10.7 Missing Int: Given an input file with four billion non-negative integers, provide an algorithm to generate an integer that is not contained in the file. Assume you have 1 GB of memory available for this task.

FOLLOW UP

What if you have only 10 MB of memory? Assume that all the values are distinct and we now have no more than one billion non-negative integers.

pg 150

SOLUTION

There are a total of 2^{32} , or 4 billion, distinct integers possible and 2^{31} non-negative integers. Therefore, we know the input file (assuming it is ints rather than longs) contains some duplicates.

We have 1 GB of memory, or 8 billion bits. Thus, with 8 billion bits, we can map all possible integers to a distinct bit with the available memory. The logic is as follows:

1. Create a bit vector (BV) with 4 billion bits. Recall that a bit vector is an array that compactly stores boolean values by using an array of ints (or another data type). Each int represents 32 boolean values.
2. Initialize BV with all 0s.
3. Scan all numbers (num) from the file and call `BV.set(num, 1)`.
4. Now scan again BV from the 0th index.
5. Return the first index which has a value of 0.

The following code demonstrates our algorithm.

```

1  long numberOfInts = ((long) Integer.MAX_VALUE) + 1;
2  byte[] bitfield = new byte [(int) (numberOfInts / 8)];
3  String filename = ...
4
5  void findOpenNumber() throws FileNotFoundException {
6      Scanner in = new Scanner(new FileReader(filename));
7      while (in.hasNextInt()) {
8          int n = in.nextInt ();
9          /* Finds the corresponding number in the bitfield by using the OR operator to
10           * set the nth bit of a byte (e.g., 10 would correspond to the 2nd bit of
11           * index 2 in the byte array). */
12          bitfield [n / 8] |= 1 << (n % 8);
13      }
14
15      for (int i = 0; i < bitfield.length; i++) {
16          for (int j = 0; j < 8; j++) {
17              /* Retrieves the individual bits of each byte. When 0 bit is found, print
18               * the corresponding value. */
19              if ((bitfield[i] & (1 << j)) == 0) {
20                  System.out.println (i * 8 + j);
21                  return;
22              }
23          }
24      }
25  }
```

Follow Up: What if we have only 10 MB memory?

It's possible to find a missing integer with two passes of the data set. We can divide up the integers into blocks of some size (we'll discuss how to decide on a size later). Let's just assume that we divide up the integers into blocks of 1000. So, block 0 represents the numbers 0 through 999, block 1 represents numbers 1000 - 1999, and so on.

Since all the values are distinct, we know how many values we *should* find in each block. So, we search through the file and count how many values are between 0 and 999, how many are between 1000 and 1999, and so on. If we count only 999 values in a particular range, then we know that a missing int must be in that range.

In the second pass, we'll actually look for which number in that range is missing. We use the bit vector approach from the first part of this problem. We can ignore any number outside of this specific range.

The question, now, is what is the appropriate block size? Let's define some variables as follows:

- Let `rangeSize` be the size of the ranges that each block in the first pass represents.
- Let `arraySize` represent the number of blocks in the first pass. Note that $\text{arraySize} = \frac{2^{31}}{\text{rangeSize}}$ since there are 2^{31} non-negative integers.

We need to select a value for `rangeSize` such that the memory from the first pass (the array) and the second pass (the bit vector) fit.

First Pass: The Array

The array in the first pass can fit in 10 megabytes, or roughly 2^{23} bytes, of memory. Since each element in the array is an int, and an int is 4 bytes, we can hold an array of at most about 2^{21} elements. So, we can deduce the following:

$$\begin{aligned}\text{arraySize} &= \frac{2^{31}}{\text{rangeSize}} \leq 2^{21} \\ \text{rangeSize} &\geq \frac{2^{31}}{2^{21}} \\ \text{rangeSize} &\geq 2^{10}\end{aligned}$$

Second Pass: The Bit Vector

We need to have enough space to store `rangeSize` bits. Since we can fit 2^{23} bytes in memory, we can fit 2^{26} bits in memory. Therefore, we can conclude the following:

$$2^{11} \leq \text{rangeSize} \leq 2^{26}$$

These conditions give us a good amount of "wiggle room," but the nearer to the middle that we pick, the less memory will be used at any given time.

The below code provides one implementation for this algorithm.

```
1  int findOpenNumber(String filename) throws FileNotFoundException {
2      int rangeSize = (1 << 20); // 2^20 bits (2^17 bytes)
3
4      /* Get count of number of values within each block. */
5      int[] blocks = getCountPerBlock(filename, rangeSize);
6
7      /* Find a block with a missing value. */
8      int blockIndex = findBlockWithMissing(blocks, rangeSize);
9      if (blockIndex < 0) return -1;
```

```

10
11  /* Create bit vector for items within this range. */
12  byte[] bitVector = getBitVectorForRange(filename, blockIndex, rangeSize);
13
14  /* Find a zero in the bit vector */
15  int offset = findZero(bitVector);
16  if (offset < 0) return -1;
17
18  /* Compute missing value. */
19  return blockIndex * rangeSize + offset;
20 }
21
22 /* Get count of items within each range. */
23 int[] getCountPerBlock(String filename, int rangeSize)
24     throws FileNotFoundException {
25     int arraySize = Integer.MAX_VALUE / rangeSize + 1;
26     int[] blocks = new int[arraySize];
27
28     Scanner in = new Scanner (new FileReader(filename));
29     while (in.hasNextInt()) {
30         int value = in.nextInt();
31         blocks[value / rangeSize]++;
32     }
33     in.close();
34     return blocks;
35 }
36
37 /* Find a block whose count is low. */
38 int findBlockWithMissing(int[] blocks, int rangeSize) {
39     for (int i = 0; i < blocks.length; i++) {
40         if (blocks[i] < rangeSize){
41             return i;
42         }
43     }
44     return -1;
45 }
46
47 /* Create a bit vector for the values within a specific range. */
48 byte[] getBitVectorForRange(String filename, int blockIndex, int rangeSize)
49     throws FileNotFoundException {
50     int startRange = blockIndex * rangeSize;
51     int endRange = startRange + rangeSize;
52     byte[] bitVector = new byte[rangeSize/Byte.SIZE];
53
54     Scanner in = new Scanner(new FileReader(filename));
55     while (in.hasNextInt()) {
56         int value = in.nextInt();
57         /* If the number is inside the block that's missing numbers, we record it */
58         if (startRange <= value && value < endRange) {
59             int offset = value - startRange;
60             int mask = (1 << (offset % Byte.SIZE));
61             bitVector[offset / Byte.SIZE] |= mask;
62         }
63     }
64     in.close();
65     return bitVector;

```

```

66 }
67
68 /* Find bit index that is 0 within byte. */
69 int findZero(byte b) {
70     for (int i = 0; i < Byte.SIZE; i++) {
71         int mask = 1 << i;
72         if ((b & mask) == 0) {
73             return i;
74         }
75     }
76     return -1;
77 }
78
79 /* Find a zero within the bit vector and return the index. */
80 int findZero(byte[] bitVector) {
81     for (int i = 0; i < bitVector.length; i++) {
82         if (bitVector[i] != ~0) { // If not all 1s
83             int bitIndex = findZero(bitVector[i]);
84             return i * Byte.SIZE + bitIndex;
85         }
86     }
87     return -1;
88 }

```

What if, as a follow up question, you are asked to solve the problem with even less memory? In this case, we can do repeated passes using the approach from the first step. We'd first check to see how many integers are found within each sequence of a million elements. Then, in the second pass, we'd check how many integers are found in each sequence of a thousand elements. Finally, in the third pass, we'd apply the bit vector.

10.8 Find Duplicates: You have an array with all the numbers from 1 to N , where N is at most 32,000. The array may have duplicate entries and you do not know what N is. With only 4 kilobytes of memory available, how would you print all duplicate elements in the array?

pg 151

SOLUTION

We have 4 kilobytes of memory which means we can address up to $8 * 4 * 2^{10}$ bits. Note that $32 * 2^{10}$ bits is greater than 32000. We can create a bit vector with 32000 bits, where each bit represents one integer.

Using this bit vector, we can then iterate through the array, flagging each element v by setting bit v to 1. When we come across a duplicate element, we print it.

```

1 void checkDuplicates(int[] array) {
2     BitSet bs = new BitSet(32000);
3     for (int i = 0; i < array.length; i++) {
4         int num = array[i];
5         int num0 = num - 1; // bitset starts at 0, numbers start at 1
6         if (bs.get(num0)) {
7             System.out.println(num);
8         } else {
9             bs.set(num0);
10        }
11    }
12 }
13
14 class BitSet {

```

```

15  int[] bitset;
16
17  public BitSet(int size) {
18      bitset = new int[(size >> 5) + 1]; // divide by 32
19  }
20
21  boolean get(int pos) {
22      int wordNumber = (pos >> 5); // divide by 32
23      int bitNumber = (pos & 0x1F); // mod 32
24      return (bitset[wordNumber] & (1 << bitNumber)) != 0;
25  }
26
27  void set(int pos) {
28      int wordNumber = (pos >> 5); // divide by 32
29      int bitNumber = (pos & 0x1F); // mod 32
30      bitset[wordNumber] |= 1 << bitNumber;
31  }
32 }

```

Note that while this isn't an especially difficult problem, it's important to implement this cleanly. This is why we defined our own bit vector class to hold a large bit vector. If our interviewer lets us (she may or may not), we could have of course used Java's built in `BitSet` class.

10.9 Sorted Matrix Search: Given an $M \times N$ matrix in which each row and each column is sorted in ascending order, write a method to find an element.

pg 151

SOLUTION

We can approach this in two ways: a more naive solution that only takes advantage of part of the sorting, and a more optimal way that takes advantage of both parts of the sorting.

Solution #1: Naive Solution

As a first approach, we can do binary search on every row to find the element. This algorithm will be $O(M \log(N))$, since there are M rows and it takes $O(\log(N))$ time to search each one. This is a good approach to mention to your interviewer before you proceed with generating a better algorithm.

To develop an algorithm, let's start with a simple example.

15	20	40	85
20	35	80	95
30	55	95	105
40	80	100	120

Suppose we are searching for the element 55. How can we identify where it is?

If we look at the start of a row or the start of a column, we can start to deduce the location. If the start of a column is greater than 55, we know that 55 can't be in that column, since the start of the column is always the minimum element. Additionally, we know that 55 can't be in any columns on the right, since the first element of each column must increase in size from left to right. Therefore, if the start of the column is greater than the element x that we are searching for, we know that we need to move further to the left.

For rows, we use identical logic. If the start of a row is bigger than x , we know we need to move upwards.

Observe that we can also make a similar conclusion by looking at the ends of columns or rows. If the end of a column or row is less than x , then we know that we must move down (for rows) or to the right (for columns) to find x . This is because the end is always the maximum element.

We can bring these observations together into a solution. The observations are the following:

- If the start of a column is greater than x , then x is to the left of the column.
- If the end of a column is less than x , then x is to the right of the column.
- If the start of a row is greater than x , then x is above that row.
- If the end of a row is less than x , then x is below that row.

We can begin in any number of places, but let's begin with looking at the starts of columns.

We need to start with the greatest column and work our way to the left. This means that our first element for comparison is `array[0][c - 1]`, where c is the number of columns. By comparing the start of columns to x (which is 55), we'll find that x must be in columns 0, 1, or 2. We will have stopped at `array[0][2]`.

This element may not be the end of a row in the full matrix, but it is an end of a row of a submatrix. The same conditions apply. The value at `array[0][2]`, which is 40, is less than 55, so we know we can move downwards.

We now have a submatrix to consider that looks like the following (the gray squares have been eliminated).

15	20	40	85
20	35	80	95
30	55	95	105
40	80	100	120

We can repeatedly apply these conditions to search for 55. Note that the only conditions we actually use are conditions 1 and 4.

The code below implements this elimination algorithm.

```
1  boolean findElement(int[][] matrix, int elem) {
2      int row = 0;
3      int col = matrix[0].length - 1;
4      while (row < matrix.length && col >= 0) {
5          if (matrix[row][col] == elem) {
6              return true;
7          } else if (matrix[row][col] > elem) {
8              col--;
9          } else {
10             row++;
11         }
12     }
13     return false;
14 }
```

Alternatively, we can apply a solution that more directly looks like binary search. The code is considerably more complicated, but it applies many of the same learnings.

Solution #2: Binary Search

Let's again look at a simple example.

15	20	70	85
20	35	80	95
30	55	95	105
40	80	100	120

We want to be able to leverage the sorting property to more efficiently find an element. So, we might ask ourselves, what does the unique ordering property of this matrix imply about where an element might be located?

We are told that every row and column is sorted. This means that element $a[i][j]$ will be greater than the elements in row i between columns 0 and $j - 1$ and the elements in column j between rows 0 and $i - 1$.

Or, in other words:

$$a[i][0] \leq a[i][1] \leq \dots \leq a[i][j-1] \leq a[i][j]$$

$$a[0][j] \leq a[1][j] \leq \dots \leq a[i-1][j] \leq a[i][j]$$

Looking at this visually, the dark gray element below is bigger than all the light gray elements.

15	20	70	85
20	35	80	95
30	55	95	105
40	80	100	120

The light gray elements also have an ordering to them: each is bigger than the elements to the left of it, as well as the elements above it. So, by transitivity, the dark gray element is bigger than the entire square.

15	20	70	85
20	35	80	95
30	55	95	105
40	80	100	120

This means that for any rectangle we draw in the matrix, the bottom right hand corner will always be the biggest.

Likewise, the top left hand corner will always be the smallest. The colors below indicate what we know about the ordering of elements (light gray < dark gray < black):

15	20	70	85
20	35	80	95
30	55	95	105
40	80	120	120

Let's return to the original problem: suppose we were searching for the value 85. If we look along the diagonal, we'll find the elements 35 and 95. What does this tell us about the location of 85?

15	20	70	85
25	35	80	95
30	55	95	105
40	80	120	120

85 can't be in the black area, since 95 is in the upper left hand corner and is therefore the smallest element in that square.

85 can't be in the light gray area either, since 35 is in the lower right hand corner of that square.

85 must be in one of the two white areas.

So, we partition our grid into four quadrants and recursively search the lower left quadrant and the upper right quadrant. These, too, will get divided into quadrants and searched.

Observe that since the diagonal is sorted, we can efficiently search it using binary search.

The code below implements this algorithm.

```
1  Coordinate findElement(int[][] matrix, Coordinate origin, Coordinate dest, int x){
2      if (!origin.inbounds(matrix) || !dest.inbounds(matrix)) {
3          return null;
4      }
5      if (matrix[origin.row][origin.column] == x) {
6          return origin;
7      } else if (!origin.isBefore(dest)) {
8          return null;
9      }
10
11     /* Set start to start of diagonal and end to the end of the diagonal. Since the
12      * grid may not be square, the end of the diagonal may not equal dest. */
13     Coordinate start = (Coordinate) origin.clone();
14     int diagDist = Math.min(dest.row - origin.row, dest.column - origin.column);
15     Coordinate end = new Coordinate(start.row + diagDist, start.column + diagDist);
16     Coordinate p = new Coordinate(0, 0);
17
18     /* Do binary search on the diagonal, looking for the first element > x */
19     while (start.isBefore(end)) {
20         p.setToAverage(start, end);
21         if (x > matrix[p.row][p.column]) {
22             start.row = p.row + 1;
23             start.column = p.column + 1;
24         } else {
25             end.row = p.row - 1;
26             end.column = p.column - 1;
27         }
28     }
29
30     /* Split the grid into quadrants. Search the bottom left and the top right. */
31     return partitionAndSearch(matrix, origin, dest, start, x);
32 }
33
34 Coordinate partitionAndSearch(int[][] matrix, Coordinate origin, Coordinate dest,
35                               Coordinate pivot, int x) {
36     Coordinate lowerLeftOrigin = new Coordinate(pivot.row, origin.column);
37     Coordinate lowerLeftDest = new Coordinate(dest.row, pivot.column - 1);
38     Coordinate upperRightOrigin = new Coordinate(origin.row, pivot.column);
39     Coordinate upperRightDest = new Coordinate(pivot.row - 1, dest.column);
40
41     Coordinate lowerLeft = findElement(matrix, lowerLeftOrigin, lowerLeftDest, x);
42     if (lowerLeft == null) {
43         return findElement(matrix, upperRightOrigin, upperRightDest, x);
44     }
```

```

45     return lowerLeft;
46 }
47
48 Coordinate findElement(int[][] matrix, int x) {
49     Coordinate origin = new Coordinate(0, 0);
50     Coordinate dest = new Coordinate(matrix.length - 1, matrix[0].length - 1);
51     return findElement(matrix, origin, dest, x);
52 }
53
54 public class Coordinate implements Cloneable {
55     public int row, column;
56     public Coordinate(int r, int c) {
57         row = r;
58         column = c;
59     }
60
61     public boolean inbounds(int[][] matrix) {
62         return row >= 0 && column >= 0 &&
63             row < matrix.length && column < matrix[0].length;
64     }
65
66     public boolean isBefore(Coordinate p) {
67         return row <= p.row && column <= p.column;
68     }
69
70     public Object clone() {
71         return new Coordinate(row, column);
72     }
73
74     public void setToAverage(Coordinate min, Coordinate max) {
75         row = (min.row + max.row) / 2;
76         column = (min.column + max.column) / 2;
77     }
78 }

```

If you read all this code and thought, “there’s no way I could do all this in an interview!” you’re probably right. You couldn’t. But, your performance on any problem is evaluated compared to other candidates on the same problem. So while you couldn’t implement all this, neither could they. You are at no disadvantage when you get a tricky problem like this.

You help yourself out a bit by separating code out into other methods. For example, by pulling `partitionAndSearch` out into its own method, you will have an easier time outlining key aspects of the code. You can then come back to fill in the body for `partitionAndSearch` if you have time.

10.10 Rank from Stream: Imagine you are reading in a stream of integers. Periodically, you wish to be able to look up the rank of a number x (the number of values less than or equal to x). Implement the data structures and algorithms to support these operations. That is, implement the method `track(int x)`, which is called when each number is generated, and the method `getRankOfNumber(int x)`, which returns the number of values less than or equal to x (not including x itself).

EXAMPLE

Stream (in order of appearance): 5, 1, 4, 4, 5, 9, 7, 13, 3

`getRankOfNumber(1)` = 0

`getRankOfNumber(3)` = 1

`getRankOfNumber(4)` = 3

pg 151

SOLUTION

A relatively easy way to implement this would be to have an array that holds all the elements in sorted order. When a new element comes in, we would need to shift the other elements to make room. Implementing `getRankOfNumber` would be quite efficient, though. We would simply perform a binary search for n , and return the index.

However, this is very inefficient for inserting elements (that is, the `track(int x)` function). We need a data structure which is good at keeping relative ordering, as well as updating when we insert new elements. A binary search tree can do just that.

Instead of inserting elements into an array, we insert elements into a binary search tree. The method `track(int x)` will run in $O(\log n)$ time, where n is the size of the tree (provided, of course, that the tree is balanced).

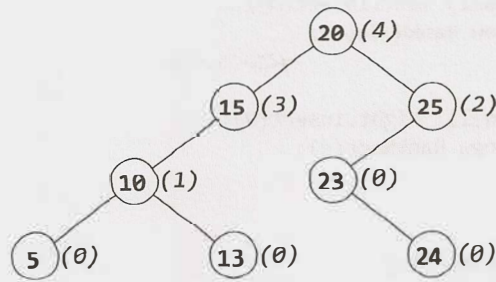
To find the rank of a number, we could do an in-order traversal, keeping a counter as we traverse. The goal is that, by the time we find x , `counter` will equal the number of elements less than x .

As long as we're moving left during searching for x , the counter won't change. Why? Because all the values we're skipping on the right side are greater than x . After all, the very smallest element (with rank of 1) is the leftmost node.

When we move to the right though, we skip over a bunch of elements on the left. All of these elements are less than x , so we'll need to increment `counter` by the number of elements in the left subtree.

Rather than counting the size of the left subtree (which would be inefficient), we can track this information as we add new elements to the tree.

Let's walk through an example on the following tree. In the below example, the value in parentheses indicates the number of nodes in the left subtree (or, in other words, the rank of the node *relative* to its subtree).



Suppose we want to find the rank of 24 in the tree above. We would compare 24 with the root, 20, and find that 24 must reside on the right. The root has 4 nodes in its left subtree, and when we include the root itself, this gives us five total nodes smaller than 24. We set counter to 5.

Then, we compare 24 with node 25 and find that 24 must be on the left. The value of counter does not update, since we're not "passing over" any smaller nodes. The value of counter is still 5.

Next, we compare 24 with node 23, and find that 24 must be on the right. Counter gets incremented by just 1 (to 6), since 23 has no left nodes.

Finally, we find 24 and we return counter: 6.

Recursively, the algorithm is the following:

```

1  int getRank(Node node, int x) {
2      if x is node.data, return node.leftSize()
3      if x is on left of node, return getRank(node.left, x)
4      if x is on right of node, return node.leftSize() + 1 + getRank(node.right, x)
5  }

```

The full code for this is below.

```

1  RankNode root = null;
2
3  void track(int number) {
4      if (root == null) {
5          root = new RankNode(number);
6      } else {
7          root.insert(number);
8      }
9  }
10
11 int getRankOfNumber(int number) {
12     return root.getRank(number);
13 }
14
15
16 public class RankNode {
17     public int left_size = 0;
18     public RankNode left, right;
19     public int data = 0;
20     public RankNode(int d) {
21         data = d;
22     }
23
24     public void insert(int d) {
25         if (d <= data) {

```

```
26         if (left != null) left.insert(d);
27         else left = new RankNode(d);
28         left_size++;
29     } else {
30         if (right != null) right.insert(d);
31         else right = new RankNode(d);
32     }
33 }
34
35 public int getRank(int d) {
36     if (d == data) {
37         return left_size;
38     } else if (d < data) {
39         if (left == null) return -1;
40         else return left.getRank(d);
41     } else {
42         int right_rank = right == null ? -1 : right.getRank(d);
43         if (right_rank == -1) return -1;
44         else return left_size + 1 + right_rank;
45     }
46 }
47 }
```

The `track` method and the `getRankOfNumber` method will both operate in $O(\log N)$ on a balanced tree and $O(N)$ on an unbalanced tree.

Note how we've handled the case in which `d` is not found in the tree. We check for the `-1` return value, and, when we find it, return `-1` up the tree. It is important that you handle cases like this.

10.11 Peaks and Valleys: In an array of integers, a “peak” is an element which is greater than or equal to the adjacent integers and a “valley” is an element which is less than or equal to the adjacent integers. For example, in the array `{5, 8, 6, 2, 3, 4, 6}`, `{8, 6}` are peaks and `{5, 2}` are valleys. Given an array of integers, sort the array into an alternating sequence of peaks and valleys.

EXAMPLE

Input: `{5, 3, 1, 2, 3}`

Output: `{5, 1, 3, 2, 3}`

pg 151

SOLUTION

Since this problem asks us to sort the array in a particular way, one thing we can try is doing a normal sort and then “fixing” the array into an alternating sequence of peaks and valleys.

Suboptimal Solution

Imagine we were given an unsorted array and then sort it to become the following:

0 1 4 7 8 9

We now have an ascending list of integers.

How can we rearrange this into a proper alternating sequence of peaks and valleys? Let's walk through it and try to do that.

- The 0 is okay.

- The 1 is in the wrong place. We can swap it with either the 0 or 4. Let's swap it with the 0.

```
1  0  4  7  8  9
```

- The 4 is okay.
- The 7 is in the wrong place. We can swap it with either the 4 or the 8. Let's swap it with the 4.

```
1  0  7  4  8  9
```

- The 9 is in the wrong place. Let's swap it with the 8.

```
1  0  7  4  9  8
```

Observe that there's nothing special about the array having these values. The relative order of the elements matters, but all sorted arrays will have the same relative order. Therefore, we can take this same approach on any sorted array.

Before coding, we should clarify the exact algorithm, though.

1. Sort the array in ascending order.
2. Iterate through the elements, starting from index 1 (not 0) and jumping two elements at a time.
3. At each element, swap it with the previous element. Since every three elements appear in the order `small <= medium <= large`, swapping these elements will always put medium as a peak: `medium <= small <= large`.

This approach will ensure that the peaks are in the right place: indexes 1, 3, 5, and so on. As long as the odd-numbered elements (the peaks) are bigger than the adjacent elements, then the even-numbered elements (the valleys) must be smaller than the adjacent elements.

The code to implement this is below.

```
1 void sortValleyPeak(int[] array) {
2     Arrays.sort(array);
3     for (int i = 1; i < array.length; i += 2) {
4         swap(array, i - 1, i);
5     }
6 }
7
8 void swap(int[] array, int left, int right) {
9     int temp = array[left];
10    array[left] = array[right];
11    array[right] = temp;
12 }
```

This algorithm runs in $O(n \log n)$ time.

Optimal Solution

To optimize past the prior solution, we need to cut out the sorting step. The algorithm must operate on an unsorted array.

Let's revisit an example.

```
9  1  0  4  8  7
```

For each element, we'll look at the adjacent elements. Let's imagine some sequences. We'll just use the numbers 0, 1 and 2. The specific values don't matter.

```
0  1  2
0  2  1    // peak
1  0  2
1  2  0    // peak
2  1  0
```

2 0 1

If the center element needs to be a peak, then two of those sequences work. Can we fix the other ones to make the center element a peak?

Yes. We can fix this sequence by swapping the center element with the largest adjacent element.

```
0 1 2 -> 0 2 1
0 2 1    // peak
1 0 2 -> 1 2 0
1 2 0    // peak
2 1 0 -> 1 2 0
2 0 1 -> 0 2 1
```

As we noted before, if we make sure the peaks are in the right place then we know the valleys are in the right place.

We should be a little cautious here. Is it possible that one of these swaps could “break” an earlier part of the sequence that we’d already processed? This is a good thing to worry about, but it’s not an issue here. If we’re swapping middle with left, then left is currently a valley. Middle is smaller than left, so we’re putting an even smaller element as a valley. Nothing will break. All is good!

The code to implement this is below.

```
1 void sortValleyPeak(int[] array) {
2     for (int i = 1; i < array.length; i += 2) {
3         int biggestIndex = maxIndex(array, i - 1, i, i + 1);
4         if (i != biggestIndex) {
5             swap(array, i, biggestIndex);
6         }
7     }
8 }
9
10 int maxIndex(int[] array, int a, int b, int c) {
11     int len = array.length;
12     int aValue = a >= 0 && a < len ? array[a] : Integer.MIN_VALUE;
13     int bValue = b >= 0 && b < len ? array[b] : Integer.MIN_VALUE;
14     int cValue = c >= 0 && c < len ? array[c] : Integer.MIN_VALUE;
15
16     int max = Math.max(aValue, Math.max(bValue, cValue));
17     if (aValue == max) return a;
18     else if (bValue == max) return b;
19     else return c;
20 }
```

This algorithm takes $O(n)$ time.

11

Solutions to Testing

11.1 Mistake: Find the mistake(s) in the following code:

```
unsigned int i;
for (i = 100; i >= 0; --i)
    printf("%d\n", i);
```

pg 157

SOLUTION

There are two mistakes in this code.

First, note that an `unsigned int` is, by definition, always greater than or equal to zero. The for loop condition will therefore always be true, and it will loop infinitely.

The correct code to print all numbers from 100 to 1, is `i > 0`. If we truly wanted to print zero, we could add an additional `printf` statement after the for loop.

```
1 unsigned int i;
2 for (i = 100; i > 0; --i)
3     printf("%d\n", i);
```

One additional correction is to use `%u` in place of `%d`, as we are printing `unsigned int`.

```
1 unsigned int i;
2 for (i = 100; i > 0; --i)
3     printf("%u\n", i);
```

This code will now correctly print the list of all numbers from 100 to 1, in descending order.

11.2 Random Crashes: You are given the source to an application which crashes when it is run. After running it ten times in a debugger, you find it never crashes in the same place. The application is single threaded, and uses only the C standard library. What programming errors could be causing this crash? How would you test each one?

pg 157

SOLUTION

The question largely depends on the type of application being diagnosed. However, we can give some general causes of random crashes.

1. *"Random Variable:"* The application may use some random number or variable component that may not be fixed for every execution of the program. Examples include user input, a random number generated by the program, or the time of day.

2. *Uninitialized Variable*: The application could have an uninitialized variable which, in some languages, may cause it to take on an arbitrary value. The values of this variable could result in the code taking a slightly different path each time.
3. *Memory Leak*: The program may have run out of memory. Other culprits are totally random for each run since it depends on the number of processes running at that particular time. This also includes heap overflow or corruption of data on the stack.
4. *External Dependencies*: The program may depend on another application, machine, or resource. If there are multiple dependencies, the program could crash at any point.

To track down the issue, we should start with learning as much as possible about the application. Who is running it? What are they doing with it? What kind of application is it?

Additionally, although the application doesn't crash in exactly the same place, it's possible that it is linked to specific components or scenarios. For example, it could be that the application never crashes if it's simply launched and left untouched, and that crashes only appear at some point after loading a file. Or, it may be that all the crashes take place within the lower level components, such as file I/O.

It may be useful to approach this by elimination. Close down all other applications on the system. Track resource use very carefully. If there are parts of the program we can disable, do so. Run it on a different machine and see if we experience the same issue. The more we can eliminate (or change), the easier we can track down the issue.

Additionally, we may be able to use tools to check for specific situations. For example, to investigate issue #2, we can utilize runtime tools which check for uninitialized variables.

These problems are as much about your brainstorming ability as they are about your approach. Do you jump all over the place, shouting out random suggestions? Or do you approach it in a logical, structured manner? Hopefully, it's the latter.

11.3 Chess Test: We have the following method used in a chess game: `boolean canMoveTo(int x, int y)`. This method is part of the `Piece` class and returns whether or not the piece can move to position `(x, y)`. Explain how you would test this method.

pg 157

SOLUTION

In this problem, there are two primary types of testing: extreme case validation (ensuring that the program doesn't crash on bad input), and general case testing. We'll start with the first type.

Testing Type #1: Extreme Case Validation

We need to ensure that the program handles bad or unusual input gracefully. This means checking the following conditions:

- Test with negative numbers for `x` and `y`
- Test with `x` larger than the width
- Test with `y` larger than the height
- Test with a completely full board
- Test with an empty or nearly empty board
- Test with far more white pieces than black

- Test with far more black pieces than white

For the error cases above, we should ask our interviewer whether we want to return false or throw an exception, and we should test accordingly.

Testing Type #2: General Testing:

General testing is much more expansive. Ideally, we would test every possible board, but there are far too many boards. We can, however, perform a reasonable coverage of different boards.

There are 6 pieces in chess, so we can test each piece against every other piece, in every possible direction. This would look something like the below code:

```
1  foreach piece a:
2      for each other type of piece b (6 types + empty space)
3          foreach direction d
4              Create a board with piece a.
5              Place piece b in direction d.
6              Try to move - check return value.
```

The key to this problem is recognizing that we can't test every possible scenario, even if we would like to. So, instead, we must focus on the essential areas.

11.4 No Test Tools: How would you load test a webpage without using any test tools?

pg 157

SOLUTION

Load testing helps to identify a web application's maximum operating capacity, as well as any bottlenecks that may interfere with its performance. Similarly, it can check how an application responds to variations in load.

To perform load testing, we must first identify the performance critical scenarios and the metrics which fulfill our performance objectives. Typical criteria include:

- Response time
- Throughput
- Resource utilization
- Maximum load that the system can bear.

Then, we design tests to simulate the load, taking care to measure each of these criteria.

In the absence of formal testing tools, we can basically create our own. For example, we could simulate concurrent users by creating thousands of virtual users. We would write a multi-threaded program with thousands of threads, where each thread acts as a real-world user loading the page. For each user, we would programmatically measure response time, data I/O, etc.

We would then analyze the results based on the data gathered during the tests and compare it with the accepted values.

11.5 Test a Pen: How would you test a pen?

pg 157

SOLUTION

This problem is largely about understanding the constraints and approaching the problem in a structured manner.

To understand the constraints, you should ask a lot of questions to understand the “who, what, where, when, how and why” of a problem (or as many of those as apply to the problem). Remember that a good tester understands exactly what he is testing before starting the work.

To illustrate the technique in this problem, let us guide you through a mock conversation.

- **Interviewer:** How would you test a pen?
- **Candidate:** Let me find out a bit about the pen. Who is going to use the pen?
- **Interviewer:** Probably children.
- **Candidate:** Okay, that’s interesting. What will they be doing with it? Will they be writing, drawing, or doing something else with it?
- **Interviewer:** Drawing.
- **Candidate:** Okay, great. On what? Paper? Clothing? Walls?
- **Interviewer:** On clothing.
- **Candidate:** Great. What kind of tip does the pen have? Felt? Ballpoint? Is it intended to wash off, or is it intended to be permanent?
- **Interviewer:** It’s intended to wash off.

Many questions later, you may get to this:

- **Candidate:** Okay, so as I understand it, we have a pen that is being targeted at 5 to 10-year-olds. The pen has a felt tip and comes in red, green, blue and black. It’s intended to wash off when clothing is washed. Is that correct?

The candidate now has a problem that is significantly different from what it initially seemed to be. This is not uncommon. In fact, many interviewers intentionally give a problem that seems clear (everyone knows what a pen is!), only to let you discover that it’s quite a different problem from what it seemed. Their belief is that users do the same thing, though users do so accidentally.

Now that you understand what you’re testing, it’s time to come up with a plan of attack. The key here is *structure*.

Consider what the different components of the object or problem, and go from there. In this case, the components might be:

- **Fact check:** Verify that the pen is felt tip and that the ink is one of the allowed colors.
- **Intended use:** Drawing. Does the pen write properly on clothing?
- **Intended use:** Washing. Does it wash off of clothing (even if it’s been there for an extended period of time)? Does it wash off in hot, warm and cold water?
- **Safety:** Is the pen safe (non-toxic) for children?
- **Unintended uses:** How else might children use the pen? They might write on other surfaces, so you need to check whether the behavior there is correct. They might also stomp on the pen, throw it, and so on.

You'll need to make sure that the pen holds up under these conditions.

Remember that in any testing question, you need to test both the intended and unintended scenarios. People don't always use the product the way you want them to.

11.6 Test an ATM: How would you test an ATM in a distributed banking system?

pg 157

SOLUTION

The first thing to do on this question is to clarify assumptions. Ask the following questions:

- Who is going to use the ATM? Answers might be "anyone," or it might be "blind people," or any number of other answers.
- What are they going to use it for? Answers might be "withdrawing money," "transferring money," "checking their balance," or many other answers.
- What tools do we have to test? Do we have access to the code, or just to the ATM?

Remember: a good tester makes sure she knows what she's testing!

Once we understand what the system looks like, we'll want to break down the problem into different testable components. These components include:

- Logging in
- Withdrawing money
- Depositing money
- Checking balance
- Transferring money

We would probably want to use a mix of manual and automated testing.

Manual testing would involve going through the steps above, making sure to check for all the error cases (low balance, new account, nonexistent account, and so on).

Automated testing is a bit more complex. We'll want to automate all the standard scenarios, as shown above, and we also want to look for some very specific issues, such as race conditions. Ideally, we would be able to set up a closed system with fake accounts and ensure that, even if someone withdraws and deposits money rapidly from different locations, he never gets money or loses money that he shouldn't.

Above all, we need to prioritize security and reliability. People's accounts must always be protected, and we must make sure that money is always properly accounted for. No one wants to unexpectedly lose money! A good tester understands the system priorities.

12

Solutions to C and C++

12.1 Last K Lines: Write a method to print the last K lines of an input file using C++.

pg 163

SOLUTION

One brute force way could be to count the number of lines (N) and then print from N-K to Nth line. But this requires two reads of the file, which is unnecessarily costly. We need a solution which allows us to read just once and be able to print the last K lines.

We can allocate an array for all K lines and the last K lines we've read in the array, and so on. Each time that we read a new line, we purge the oldest line from the array.

But—you might ask—wouldn't this require shifting elements in the array, which is also very expensive? No, not if we do it correctly. Instead of shifting the array each time, we will use a circular array.

With a circular array, we always replace the oldest item when we read a new line. The oldest item is tracked in a separate variable, which adjusts as we add new items.

The following is an example of a circular array:

```
step 1 (initially): array = {a, b, c, d, e, f}. p = 0
step 2 (insert g): array = {g, b, c, d, e, f}. p = 1
step 3 (insert h): array = {g, h, c, d, e, f}. p = 2
step 4 (insert i): array = {g, h, i, d, e, f}. p = 3
```

The code below implements this algorithm.

```
1 void printLast10Lines(char* fileName) {
2     const int K = 10;
3     ifstream file (fileName);
4     string L[K];
5     int size = 0;
6
7     /* read file line by line into circular array */
8     /* peek() so an EOF following a line ending is not considered a separate line */
9     while (file.peek() != EOF) {
10         getline(file, L[size % K]);
11         size++;
12     }
13
14     /* compute start of circular array, and the size of it */
15     int start = size > K ? (size % K) : 0;
16     int count = min(K, size);
17 }
```

```

18  /* print elements in the order they were read */
19  for (int i = 0; i < count; i++) {
20      cout << L[(start + i) % K] << endl;
21  }
22  }

```

This solution will require reading in the whole file, but only ten lines will be in memory at any given point.

12.2 Reverse String: Implement a function `void reverse(char* str)` in C or C++ which reverses a null-terminated string.

pg 163

SOLUTION

This is a classic interview question. The only “gotcha” is to try to do it in place, and to be careful for the null character.

We will implement this in C.

```

1  void reverse(char *str) {
2      char* end = str;
3      char tmp;
4      if (str) {
5          while (*end) { /* find end of the string */
6              ++end;
7          }
8          --end; /* set one char back, since last char is null */
9
10         /* swap characters from start of string with the end of the string, until the
11          * pointers meet in middle. */
12         while (str < end) {
13             tmp = *str;
14             *str++ = *end;
15             *end-- = tmp;
16         }
17     }
18 }

```

This is just one of many ways to implement this solution. We could even implement this code recursively (but we wouldn’t recommend it).

12.3 Hash Table vs STL Map: Compare and contrast a hash table and an STL map. How is a hash table implemented? If the number of inputs is small, which data structure options can be used instead of a hash table?

pg 163

SOLUTION

In a hash table, a value is stored by calling a hash function on a key. Values are not stored in sorted order. Additionally, since hash tables use the key to find the index that will store the value, an insert or lookup can be done in amortized $O(1)$ time (assuming few collisions in the hash table). In a hash table, one must also handle potential collisions. This is often done by chaining, which means to create a linked list of all the values whose keys map to a particular index.

An STL map inserts the key/value pairs into a binary search tree based on the keys. There is no need to handle collisions, and, since the tree is balanced, the insert and lookup time is guaranteed to be $O(\log N)$.

How is a hash table implemented?

A hash table is traditionally implemented with an array of linked lists. When we want to insert a key/value pair, we map the key to an index in the array using a hash function. The value is then inserted into the linked list at that position.

Note that the elements in a linked list at a particular index of the array do not have the same key. Rather, `hashFunction(key)` is the same for these values. Therefore, in order to retrieve the value for a specific key, we need to store in each node both the exact key and the value.

To summarize, the hash table will be implemented with an array of linked lists, where each node in the linked list holds two pieces of data: the value and the original key. In addition, we will want to note the following design criteria:

1. We want to use a good hash function to ensure that the keys are well distributed. If they are not well distributed, then we would get a lot of collisions and the speed to find an element would decline.
2. No matter how good our hash function is, we will still have collisions, so we need a method for handling them. This often means chaining via a linked list, but it's not the only way.
3. We may also wish to implement methods to dynamically increase or decrease the hash table size depending on capacity. For example, when the ratio of the number of elements to the table size exceeds a certain threshold, we may wish to increase the hash table size. This would mean creating a new hash table and transferring the entries from the old table to the new table. Because this is an expensive operation, we want to be careful to not do it too often.

What can be used instead of a hash table, if the number of inputs is small?

You can use an STL map or a binary tree. Although this takes $O(\log(n))$ time, the number of inputs may be small enough to make this time negligible.

12.4 Virtual Functions: How do virtual functions work in C++?

pg 164

SOLUTION

A virtual function depends on a "vtable" or "Virtual Table." If any function of a class is declared to be virtual, a vtable is constructed which stores addresses of the virtual functions of this class. The compiler also adds a hidden `vp_ptr` variable in all such classes which points to the vtable of that class. If a virtual function is not overridden in the derived class, the vtable of the derived class stores the address of the function in its parent class. The vtable is used to resolve the address of the function when the virtual function is called. Dynamic binding in C++ is performed through the vtable mechanism.

Thus, when we assign the derived class object to the base class pointer, the `vp_ptr` variable points to the vtable of the derived class. This assignment ensures that the most derived virtual function gets called.

Consider the following code.

```
1 class Shape {
2     public:
3         int edge_length;
4         virtual int circumference () {
```

```

5      cout << "Circumference of Base Class\n";
6      return 0;
7  }
8  };
9
10 class Triangle: public Shape {
11     public:
12     int circumference () {
13         cout<< "Circumference of Triangle Class\n";
14         return 3 * edge_length;
15     }
16 };
17
18 void main() {
19     Shape * x = new Shape();
20     x->circumference(); // "Circumference of Base Class"
21     Shape *y = new Triangle();
22     y->circumference(); // "Circumference of Triangle Class"
23 }

```

In the previous example, `circumference` is a virtual function in the `Shape` class, so it becomes virtual in each of the derived classes (`Triangle`, etc). C++ non-virtual function calls are resolved at compile time with static binding, while virtual function calls are resolved at runtime with dynamic binding.

12.5 Shallow vs Deep Copy: What is the difference between deep copy and shallow copy? Explain how you would use each.

pg 164

SOLUTION

A shallow copy copies all the member values from one object to another. A deep copy does all this and also deep copies any pointer objects.

An example of shallow and deep copy is below.

```

1  struct Test {
2      char * ptr;
3  };
4
5  void shallow_copy(Test & src, Test & dest) {
6      dest.ptr = src.ptr;
7  }
8
9  void deep_copy(Test & src, Test & dest) {
10     dest.ptr = (char*)malloc(strlen(src.ptr) + 1);
11     strcpy(dest.ptr, src.ptr);
12 }

```

Note that `shallow_copy` may cause a lot of programming runtime errors, especially with the creation and deletion of objects. Shallow copy should be used very carefully and only when a programmer really understands what he wants to do. In most cases, shallow copy is used when there is a need to pass information about a complex structure without actual duplication of data. One must also be careful with destruction of objects in a shallow copy.

In real life, shallow copy is rarely used. Deep copy should be used in most cases, especially when the size of the copied structure is small.

12.6 Volatile: What is the significance of the keyword "volatile" in C?

pg 164

SOLUTION

The keyword `volatile` informs the compiler that the value of variable it is applied to can change from the outside, without any update done by the code. This may be done by the operating system, the hardware, or another thread. Because the value can change unexpectedly, the compiler will therefore reload the value each time from memory.

A volatile integer can be declared by either of the following statements:

```
int volatile x;  
volatile int x;
```

To declare a pointer to a volatile integer, we do the following:

```
volatile int * x;  
int volatile * x;
```

A volatile pointer to non-volatile data is rare, but can be done.

```
int * volatile x;
```

If you wanted to declare a volatile variable pointer for volatile memory (both pointer address and memory contained are volatile), you would do the following:

```
int volatile * volatile x;
```

Volatile variables are not optimized, which can be very useful. Imagine this function:

```
1 int opt = 1;  
2 void Fn(void) {  
3     start:  
4     if (opt == 1) goto start;  
5     else break;  
6 }
```

At first glance, our code appears to loop infinitely. The compiler may try to optimize it to:

```
1 void Fn(void) {  
2     start:  
3     int opt = 1;  
4     if (true)  
5     goto start;  
6 }
```

This becomes an infinite loop. However, an external operation might write '0' to the location of variable `opt`, thus breaking the loop.

To prevent the compiler from performing such optimization, we want to signal that another element of the system could change the variable. We do this using the `volatile` keyword, as shown below.

```
1 volatile int opt = 1;  
2 void Fn(void) {  
3     start:  
4     if (opt == 1) goto start;  
5     else break;  
6 }
```

Volatile variables are also useful when multi-threaded programs have global variables and any thread can modify these shared variables. We may not want optimization on these variables.

12.7 Virtual Base Class: Why does a destructor in base class need to be declared virtual?

pg 164

SOLUTION

Let's think about why we have virtual methods to start with. Suppose we have the following code:

```

1  class Foo {
2      public:
3          void f();
4      };
5
6  class Bar : public Foo {
7      public:
8          void f();
9      }
10
11  Foo * p = new Bar();
12  p->f();

```

Calling `p->f()` will result in a call to `Foo::f()`. This is because `p` is a pointer to `Foo`, and `f()` is not virtual.

To ensure that `p->f()` will invoke the most derived implementation of `f()`, we need to declare `f()` to be a virtual function.

Now, let's go back to our destructor. Destructors are used to clean up memory and resources. If `Foo`'s destructor were not virtual, then `Foo`'s destructor would be called, even when `p` is *really* of type `Bar`.

This is why we declare destructors to be virtual; we want to ensure that the destructor for the most derived class is called.

12.8 Copy Node: Write a method that takes a pointer to a `Node` structure as a parameter and returns a complete copy of the passed in data structure. The `Node` data structure contains two pointers to other `Nodes`.

pg 164

SOLUTION

The algorithm will maintain a mapping from a node address in the original structure to the corresponding node in the new structure. This mapping will allow us to discover previously copied nodes during a traditional depth-first traversal of the structure. Traversals often mark visited nodes—the mark can take many forms and does not necessarily need to be stored in the node.

Thus, we have a simple recursive algorithm:

```

1  typedef map<Node*, Node*> NodeMap;
2
3  Node * copy_recursive(Node * cur, NodeMap & nodeMap) {
4      if (cur == NULL) {
5          return NULL;
6      }
7
8      NodeMap::iterator i = nodeMap.find(cur);
9      if (i != nodeMap.end()) {
10         // we've been here before, return the copy
11         return i->second;
12     }

```

```

13
14     Node * node = new Node;
15     nodeMap[cur] = node; // map current before traversing links
16     node->ptr1 = copy_recursive(cur->ptr1, nodeMap);
17     node->ptr2 = copy_recursive(cur->ptr2, nodeMap);
18     return node;
19 }
20
21 Node * copy_structure(Node * root) {
22     NodeMap nodeMap; // we will need an empty map
23     return copy_recursive(root, nodeMap);
24 }

```

12.9 Smart Pointer: Write a smart pointer class. A smart pointer is a data type, usually implemented with templates, that simulates a pointer while also providing automatic garbage collection. It automatically counts the number of references to a *SmartPointer*<T*> object and frees the object of type T when the reference count hits zero.

pg 164

SOLUTION

A smart pointer is the same as a normal pointer, but it provides safety via automatic memory management. It avoids issues like dangling pointers, memory leaks and allocation failures. The smart pointer must maintain a single reference count for all references to a given object.

This is one of those problems that seems at first glance pretty overwhelming, especially if you're not a C++ expert. One useful way to approach the problem is to divide the problem into two parts: (1) outline the pseudocode and approach and then (2) implement the detailed code.

In terms of the approach, we need a reference count variable that is incremented when we add a new reference to the object and decremented when we remove a reference. The code should look something like the below pseudocode:

```

1  template <class T> class SmartPointer {
2      /* The smart pointer class needs pointers to both the object itself and to the
3       * ref count. These must be pointers, rather than the actual object or ref count
4       * value, since the goal of a smart pointer is that the reference count is
5       * tracked across multiple smart pointers to one object. */
6      T * obj;
7      unsigned * ref_count;
8  }

```

We know we need constructors and a single destructor for this class, so let's add those first.

```

1  SmartPointer(T * object) {
2      /* We want to set the value of T * obj, and set the reference counter to 1. */
3  }
4
5  SmartPointer(SmartPointer<T>& sptr) {
6      /* This constructor creates a new smart pointer that points to an existing
7       * object. We will need to first set obj and ref_count to pointer to sptr's obj
8       * and ref_count. Then, because we created a new reference to obj, we need to
9       * increment ref_count. */
10 }
11
12 ~SmartPointer(SmartPointer<T> sptr) {
13     /* We are destroying a reference to the object. Decrement ref_count. If

```

```

14     * ref_count is 0, then free the memory created by the integer and destroy the
15     * object. */
16 }

```

There's one additional way that references can be created: by setting one `SmartPointer` equal to another. We'll want to override the `equal` operator to handle this, but for now, let's sketch the code like this.

```

1  onSetEquals(SmartPointer<T> ptr1, SmartPointer<T> ptr2) {
2      /* If ptr1 has an existing value, decrement its reference count. Then, copy the
3       * pointers to obj and ref_count over. Finally, since we created a new
4       * reference, we need to increment ref_count. */
5  }

```

Getting just the approach, even without filling in the complicated C++ syntax, would count for a lot. Finishing out the code is now just a matter of filling in the details.

```

1  template <class T> class SmartPointer {
2      public:
3          SmartPointer(T * ptr) {
4              ref = ptr;
5              ref_count = (unsigned*)malloc(sizeof(unsigned));
6              *ref_count = 1;
7          }
8
9          SmartPointer(SmartPointer<T> & sptr) {
10             ref = sptr.ref;
11             ref_count = sptr.ref_count;
12             ++(*ref_count);
13         }
14
15         /* Override the equal operator, so that when you set one smart pointer equal to
16          * another the old smart pointer has its reference count decremented and the new
17          * smart pointer has its reference count incremented. */
18         SmartPointer<T> & operator=(SmartPointer<T> & sptr) {
19             if (this == &sptr) return *this;
20
21             /* If already assigned to an object, remove one reference. */
22             if (*ref_count > 0) {
23                 remove();
24             }
25
26             ref = sptr.ref;
27             ref_count = sptr.ref_count;
28             ++(*ref_count);
29             return *this;
30         }
31
32         ~SmartPointer() {
33             remove(); // Remove one reference to object.
34         }
35
36         T getValue() {
37             return *ref;
38         }
39
40     protected:
41         void remove() {
42             --(*ref_count);
43             if (*ref_count == 0) {

```